

b)

- Planes de Prueba:

¿Qué es un plan de pruebas de Software? ¿Cuál es el ciclo de vida de pruebas de Software? Realice un gráfico del ciclo de vida.

Un plan de pruebas de software es un documento que guía, organiza y controla todas las actividades de validación y control de calidad en un proyecto de desarrollo de software. Esto quiere decir que define cómo se garantizará la calidad del producto.

Sirve como guía de referencia para el equipo de QA/testing y como acuerdo entre los interesados sobre qué se va a probar, cómo, cuándo y con qué recursos.

Algunos elementos típicos son:

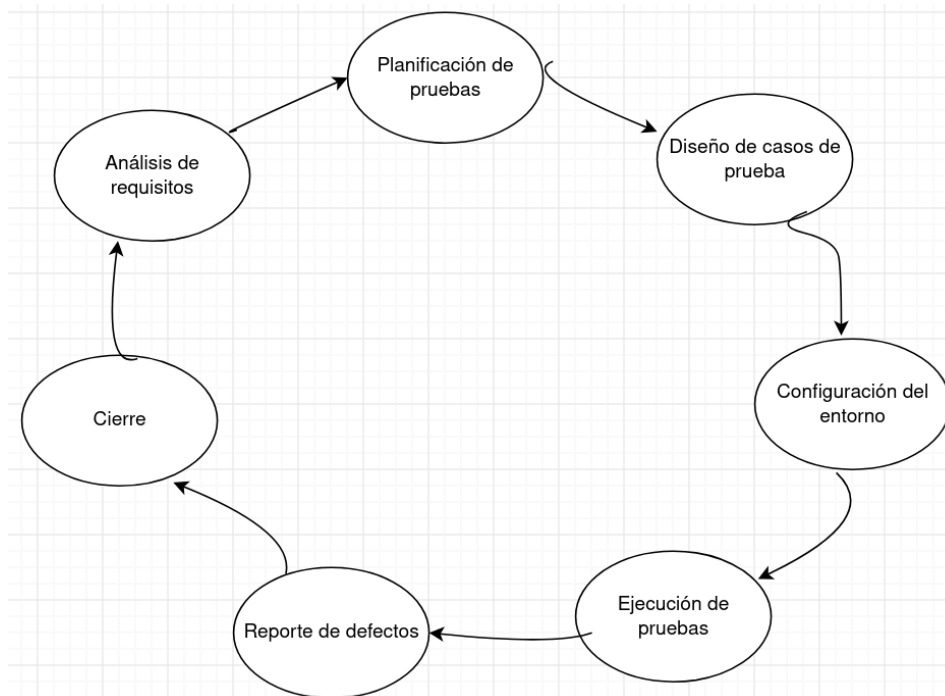
1. **Objetivo y alcance:** qué funcionalidades se probarán y cuáles quedan fuera.
2. **Criterios de entrada y salida:** condiciones que deben cumplirse para comenzar y para dar por finalizadas las pruebas.
3. **Estrategia de pruebas:** tipos de prueba a aplicar (funcionales, no funcionales, regresión, etc.) y niveles (unitarias, integración, sistema, aceptación).
4. **Recursos:** equipo humano, herramientas, entornos, hardware/software necesario.
5. **Cronograma:** fechas y duración estimada de cada fase.
6. **Casos de prueba:** escenarios que se ejecutarán, con datos de entrada y resultados esperados.
7. **Riesgos y contingencias:** posibles problemas y cómo mitigarlos.
8. **Criterios de aceptación:** condiciones para considerar el software apto para producción.
9. **Roles y responsabilidades:** quién hace qué dentro del proceso.

El ciclo de vidas de prueba es la secuencia de fases que sigue el proceso de pruebas, generalmente en paralelo al ciclo de vida de desarrollo. Sus fases secuenciales son:

1. **Análisis de requisitos:** estudio de los requerimientos funcionales y no funcionales para entender qué se debe probar.

2. **Planificación de pruebas:** se elabora el plan de pruebas: estrategia, alcance, estimaciones de tiempo y costo, herramientas y recursos necesarios.
3. **Diseño de casos de prueba:** se crean los casos de prueba, guiones (scripts) y datos de prueba en base a los requisitos. Se prioriza qué probar primero según riesgo.
4. **Configuración del entorno de pruebas:** se preparan los servidores, redes y bases de datos para simular con el entorno del usuario final donde se ejecutarán las pruebas.
5. **Ejecución de pruebas:** se corren los escenarios previamente diseñados y se comparan los resultados obtenidos con los esperados.
6. **Reporte y seguimiento de defectos:** si se encuentran discrepancias (bugs), se documentan y envían a desarrollo para su corrección, y posteriormente se re-testean los arreglos.
7. **Cierre de pruebas:** se evalúa si se cumplieron los objetivos, se documentan las lecciones aprendidas y se emite el informe final de métricas de calidad.

Gráfico del ciclo de vida:



- Tipos de Prueba:

Realizar un cuadro con los tipos de prueba, donde se describa cada uno y especifique el objetivo de su ejecución.

Tipo de Prueba	Descripción	Objetivo de Ejecución
Pruebas Unitarias	Evaluación aislada del componente de software o unidad de código más pequeña posible, como funciones, métodos o clases individuales, simulando dependencias externas mediante dobles de prueba.	Comprobar que cada unidad lógica funcione de manera correcta individualmente según su especificación y detectar defectos en etapas tempranas del desarrollo.
Pruebas de Integración	Verificación de la comunicación, interacción e interfaces entre múltiples módulos, componentes o subsistemas cuando operan en conjunto.	Descubrir defectos en los protocolos de comunicación e intercambio de datos entre los distintos módulos del sistema.
Pruebas del Sistema	Ejecución y análisis del comportamiento global del producto completo e integrado en un entorno que emula fielmente las condiciones operativas de producción.	Evaluar el cumplimiento de los requerimientos funcionales y no funcionales especificados para la solución global.
Pruebas de Aceptación	Validación formal del producto final ejecutada por el usuario final, cliente o partes interesadas de negocio en escenarios operativos representativos.	Determinar si el software satisface las necesidades del negocio y los criterios de aceptación del cliente.
Pruebas de Regresión	Reejecución sistemática de conjuntos de pruebas funcionales o no funcionales tras incorporar modificaciones, parches o nuevas funcionalidades en la base de código.	Garantizar que las adiciones recientes al código no hayan introducido nuevos defectos ni deteriorado funcionalidades ya implementadas.

Pruebas de Rendimiento	Evaluación cuantitativa de la velocidad de respuesta, estabilidad, uso de recursos y escalabilidad del sistema bajo diferentes volúmenes de carga de trabajo.	Identificar cuellos de botella de infraestructura, tiempos de latencia y verificar si el sistema cumple con los umbrales de desempeño requeridos ante alta demanda.
Pruebas de Seguridad	Análisis estructurado para examinar vulnerabilidades, mecanismos de cifrado, controles de acceso y tolerancia a vectores de explotación maliciosos.	Prevenir accesos no autorizados, proteger la confidencialidad e integridad de la información y mitigar posibles riesgos de intrusión.
Pruebas de Usabilidad	Medición empírica de la interacción directa del usuario con la interfaz de la aplicación, evaluando facilidad de aprendizaje, intuición y nivel de esfuerzo requerido.	Verificar que el sistema ofrezca una experiencia de usuario clara, accesible y eficiente alineadas con el objetivo del software.

- Patrones de Automatización de Pruebas.

¿Qué son los patrones de automatización de pruebas de software (Modelo de Objeto de Páginas “POM”, Pruebas Basadas en Datos, Desarrollo Impulsado por Comportamiento “BDD”, Pruebas de Carga y Pruebas de Stress)? Realizar una explicación de cada uno de ellos.

POM, Pruebas Basadas en Datos (DDT) y BDD son patrones de diseño o metodologías sobre cómo se escribe y estructura el código de las pruebas. Por otro lado, las Pruebas de Carga y Stress son *tipos* específicos de pruebas de rendimiento que evalúan el comportamiento del sistema bajo presión.

POM (Page Object Model): Patrón de diseño utilizado principalmente en la automatización de pruebas de interfaces de usuario (UI). Su principio básico es crear un repositorio de objetos donde **cada página web de la aplicación se representa mediante una "Clase"** en el código. Los elementos de esa página y las acciones que se pueden hacer en ella se definen como variables y métodos dentro de esa clase.

DDT (Data Driven Testing): Enfoque donde la lógica del script de prueba se separa por completo de los datos que se van a probar. En lugar de escribir los valores de prueba directamente dentro del código, los datos se extraen de fuentes externas como archivos Excel, CSV, bases de datos o archivos JSON. Ejemplo: se escribe un único script de prueba para la

función de login. Luego, se conecta ese script a un archivo Excel que tiene 100 filas con diferentes combinaciones de usuario y contraseña. El script se ejecutará 100 veces, inyectando una fila de datos diferente en cada iteración.

BDD (Behavior-Driven Development): No es solo un patrón de automatización, sino una metodología ágil que fomenta la colaboración entre desarrolladores, testers y la parte de negocio. Busca que todos hablen un "lenguaje común" que sea fácil de entender para personas sin conocimientos técnicos. Los requisitos se escriben en un formato estructurado de lenguaje natural, comúnmente usando la sintaxis **Gherkin** con las palabras clave *Given*, *When*, *Then* (Dado, Cuando, Entonces) y herramientas como **Cucumber** o **SpecFlow** leen estas frases en texto plano y las mapean a código de prueba real en el backend.

Pruebas de Carga: La prueba de carga consiste en someter a un sistema informático, servidor o red a un nivel de tráfico de usuarios que se considera normal o dentro del máximo esperado. El objetivo principal de esta prueba no es romper el sistema, sino verificar su rendimiento y estabilidad en condiciones de uso realistas. Se busca asegurar que, bajo la carga de trabajo para la cual el software fue diseñado, los tiempos de respuesta sigan siendo rápidos, no se produzcan cuellos de botella y la experiencia del usuario final no se degrade.

Pruebas de Stress: La prueba de stress lleva al sistema al extremo. Consiste en inyectar una carga de trabajo muy superior al límite de diseño, incrementando la presión progresivamente hasta provocar un fallo. El objetivo principal es encontrar el punto de quiebre de la arquitectura técnica. No se trata de evaluar si funciona bien, sino de descubrir exactamente en qué momento deja de funcionar y, lo más importante, cómo reacciona al fallar. Esta prueba permite observar si el sistema colapsa de manera segura (sin corromper o perder información), qué componente es el primero en caer (la memoria, la CPU, la base de datos) y si es capaz de recuperarse automáticamente una vez que el nivel de tráfico vuelve a bajar.